

PySpark RDD – Theoretical Assignment Notes

1. Difference between map() and flatMap()

What the question is asking

Understand how map() and flatMap() transform RDD elements and how their output structure differs.

Direct answer

map() applies a function to each element and produces one output for each input. **flatMap()** can produce multiple outputs for one input and flattens them into one RDD.

Concept explained in simple terms

Simple concept: map = one input → one output. flatMap = one input → zero, one, or many outputs. For example, a sentence can become several words with flatMap().

Technical/code example

```
sentences = ["spark is fast", "spark is powerful"]
rdd = sc.parallelize(sentences)

r1 = rdd.map(lambda x: x.split())
# [['spark', 'is', 'fast'], ['spark', 'is', 'powerful']]

r2 = rdd.flatMap(lambda x: x.split())
# ['spark', 'is', 'fast', 'spark', 'is', 'powerful']
```

Visual / flow

Sentences RDD → map() → RDD of lists

Sentences RDD → flatMap() → flat RDD of words

Important points to remember

map() does not flatten returned collections; flatMap() does. Both are transformations and return RDDs.

2. Difference between groupByKey() and reduceByKey()

What the question is asking

Understand how the two operations handle values belonging to the same key.

Direct answer

groupByKey() groups all values for the same key. **reduceByKey()** combines values for the same key using a reduction function such as addition.

Concept explained in simple terms

Simple concept: groupByKey keeps the grouped values; reduceByKey produces one combined value per key. For aggregation, reduceByKey is generally preferred because it can combine values during processing and reduce shuffle data.

Technical/code example

```
data = [("IT", 80000), ("HR", 60000),
        ("IT", 90000), ("HR", 65000)]
rdd = sc.parallelize(data)

grouped = rdd.groupByKey().mapValues(list)
# IT → [80000, 90000], HR → [60000, 65000]

totals = rdd.reduceByKey(lambda a, b: a + b)
# IT → 170000, HR → 125000
```

Visual / flow

Key-value RDD → groupByKey() → key → all values

Key-value RDD → reduceByKey(+) → key → combined value

Important points to remember

Use groupByKey() when all values are needed. Use reduceByKey() for aggregation such as sum, count, min, or max.

3. What is coalesce()?

What the question is asking

Understand how Spark reduces the number of partitions of an RDD.

Direct answer

coalesce() reduces the number of partitions of an RDD. It is commonly used to reduce partitions after filtering or to reduce the number of output part files.

Concept explained in simple terms

Simple concept: if an RDD has 10 partitions, `coalesce(2)` reduces it to 2 partitions. It is mainly intended for reducing partitions; `repartition()` is generally used when increasing or freely reshuffling the partition count.

Technical/code example

```
rdd = sc.parallelize(range(1, 101), 10)
print(rdd.getNumPartitions()) # 10

smaller = rdd.coalesce(2)
print(smaller.getNumPartitions()) # 2
```

Visual / flow

10 partitions → `coalesce(2)` → 2 partitions

Important points to remember

`coalesce()` reduces partitions. Fewer partitions can mean fewer part-* files when saving an RDD.

4. What are PySpark transformations? How do transformations work?

What the question is asking

Understand what a transformation is, how it creates a new RDD, and why transformations are lazily evaluated.

Direct answer

A **transformation** is an RDD operation that creates a new RDD from an existing RDD. Transformations are **lazily evaluated**: Spark records the operations and normally executes them when an action is called.

Concept explained in simple terms

Simple concept: `map()`, `filter()`, `flatMap()`, `distinct()`, `groupByKey()`, and `reduceByKey()` are examples. An action such as `collect()`, `count()`, `sum()`, `max()`, `min()`, or `saveAsTextFile()` triggers the required computation.

Technical/code example

```
rdd = sc.parallelize([1, 2, 3, 4, 5])

result = rdd.filter(lambda x: x > 2) \
    .map(lambda x: x * 10)

print(result.collect())
# [30, 40, 50]
```

Visual / flow

Original RDD → transformations → new RDD → action → execution/result

Important points to remember

Transformations create RDDs and are lazy. Actions trigger execution and return a result or save data.

5. Narrow vs. Wide Transformations

What the question is asking

Understand whether a transformation can process data within existing partitions or must redistribute data between partitions.

Direct answer

Narrow transformations do not require data to be redistributed across partitions; each output partition depends on a limited/local set of input partitions. **Wide transformations** require redistribution of data between partitions, causing a **shuffle**.

Concept explained in simple terms

Simple concept: `map()`, `filter()`, and `flatMap()` can generally process each partition independently. `reduceByKey()`, `groupByKey()`, and `distinct()` require matching/related records to be brought together, so Spark performs a shuffle.

Technical/code example

```
rdd = sc.parallelize([1, 2, 3, 4, 5, 6], 3)
doubled = rdd.map(lambda x: x * 2) # narrow

pairs = sc.parallelize([("IT",80), ("HR",60),
                        ("IT",90), ("HR",65)], 2)
totals = pairs.reduceByKey(lambda a, b: a + b) # wide
```

Visual / flow

Narrow: P1 → P1, P2 → P2, P3 → P3 (no shuffle)

Wide: P1/P2/P3 → shuffle → target partitions

Important points to remember

Typical narrow: `map()`, `filter()`, `flatMap()`. Typical wide: `groupByKey()`, `reduceByKey()`, `distinct()`. Wide transformations are generally more expensive because of shuffle.